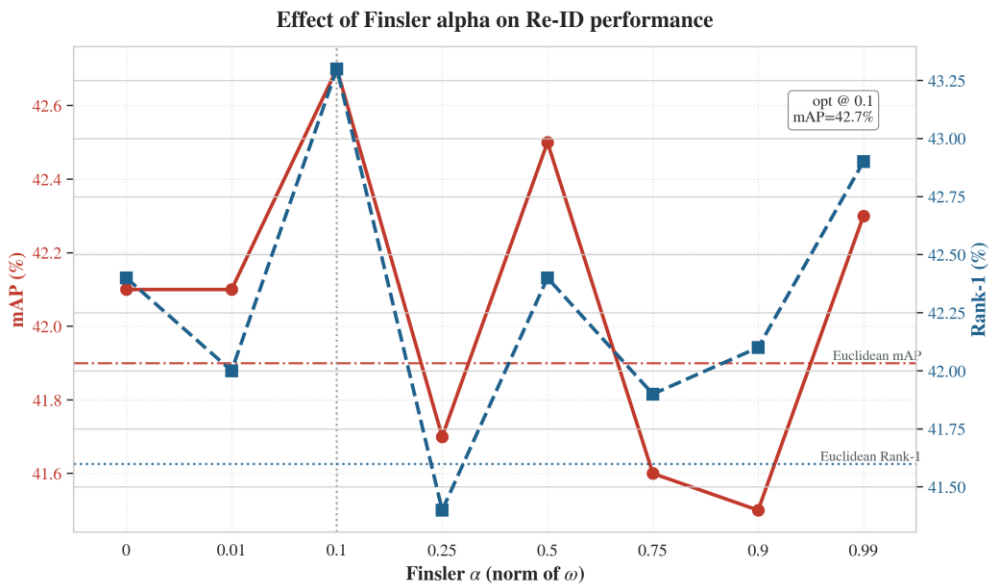
Previously

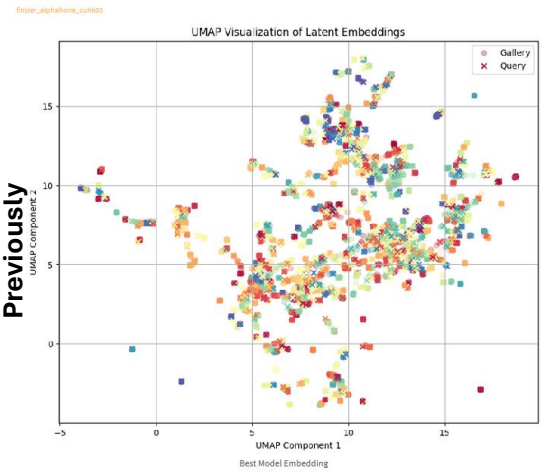


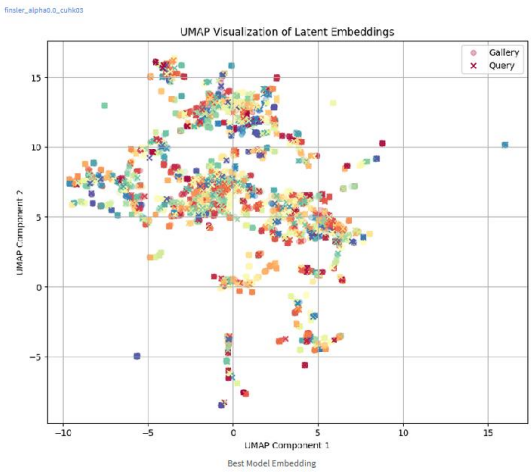
- Alpha 0.0 and alpha=None have different compute graphs, which can change behavior in kernel fusion and rounding under autocast
- This creates tiny numeric differences that accumulate over time
- Non-deterministic behavior from cudnn.benchmark=True

- Do five runs per configuration and average it out (smoothen out to make general configurations from it)
- Ablate the losses, maybe finsler affects this component most
- Maybe other components the model we can finslerize

```
# Base hyper-parameters (shared across jobs)
ARCH=resnet50            # Default
BATCH_SIZE=256           # Default
LAM=1.5                  # Default
KNN=10                   # Default
ITERS=500                # Default
WARMUP_STEP=10           # Default
EPOCHS=60                # Default
```

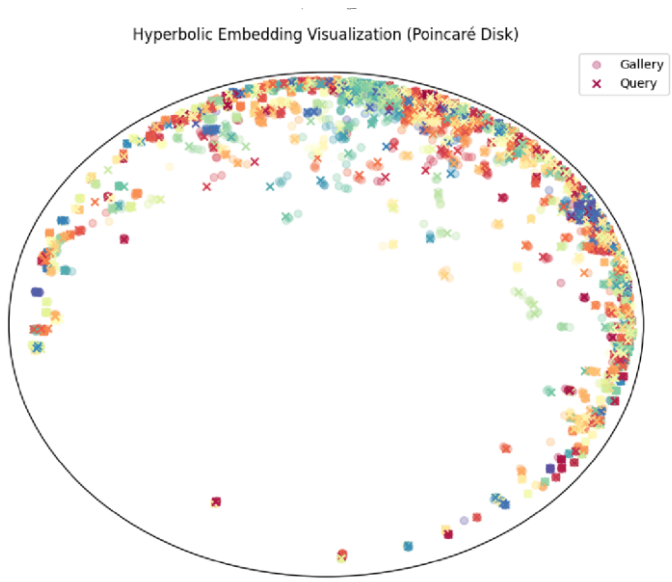
Previously





- Seed UMAP + Remove multiple processes that uses non-deterministic (some parallel things, e.g., number of jobs)
- Seed before UMAP and seed in UMAP itself
- Maybe use TSNE and seed everything
- Use ISOMAP (less likely to form cluster)
- Play with the hyperparameters
- Subsample the clusters think more about what I want to show

Previously



- Maybe PCA preprocessing
- See how imagenet embeds things
- There should be some visual clusters
- But here there are no clusters whatsoever
- 2000 embeddings should be okay

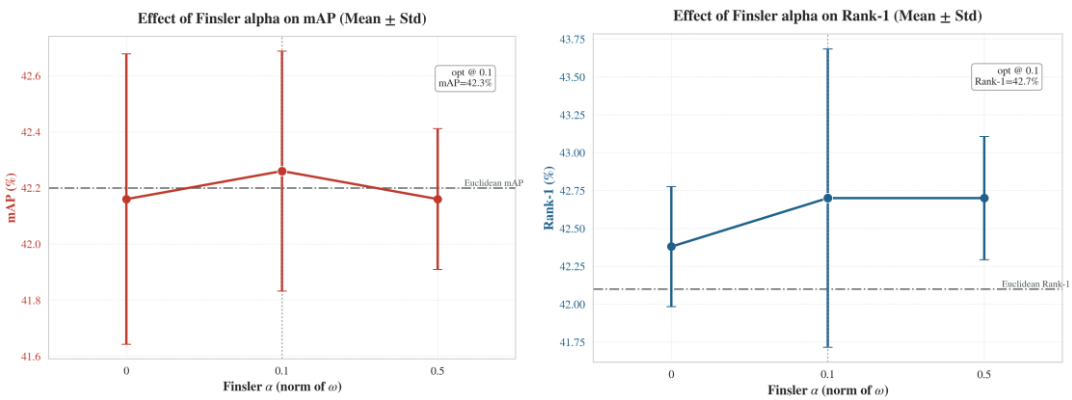
Previously

ID	Loss Function Configuration	Justification / What we want to show
1	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{ce}}^{orig}$	**Sanity Check**: Values should match between Euclidean and Hyperbolic variants*
2	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{triplet}}^{orig}$	Isolates the impact of the triplet loss + check training stability
3	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{align}}$	Isolates the impact of alignment loss (See Eq. 5) + check training stability
4	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{uniform}}$	Isolates the impact of uniformity loss (See Eq. 6) + check training stability
5	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{domain}}$	Isolates the impact of domain loss (See Eq. 7) + check training stability
6	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{ce}}^{orig} + \mathcal{L}_{\text{triplet}}^{orig}$	**Baseline w/o augmentations**: Isolates the effect of hyperbolic geometry on standard person Re-ID Training
7	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{ce}}^{orig} + \mathcal{L}_{\text{triplet}}^{orig} + \mathcal{L}_{\text{ce}}^{aug}$	Cross-entropy on augmented images using same labels as originals
8	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{ce}}^{orig} + \mathcal{L}_{\text{triplet}}^{orig} + \mathcal{L}_{\text{align}}$	Alignment between original and augmented hyperbolic embeddings (See Eq. 5)
9	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{ce}}^{orig} + \mathcal{L}_{\text{triplet}}^{orig} + \mathcal{L}_{\text{uniform}}$	Uniformity of hyperbolic embeddings on the Poincaré ball (See Eq. 6)
10	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{ce}}^{orig} + \mathcal{L}_{\text{triplet}}^{orig} + \mathcal{L}_{\text{domain}}$	Domain-specific uniformity with prototypes defined and updated in the Poincaré ball (See Eq. 8)
11	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{ce}}^{orig} + \mathcal{L}_{\text{triplet}}^{orig} + \mathcal{L}_{\text{align}} + \mathcal{L}_{\text{uniform}}$	Synergy between $\mathcal{L}_{\text{align}}$ and $\mathcal{L}_{\text{uniform}}$ as regularizers; impact of $\mathcal{L}_{\text{domain}}$
12	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{ce}}^{orig} + \mathcal{L}_{\text{triplet}}^{orig} + \mathcal{L}_{\text{align}} + \mathcal{L}_{\text{domain}}$	Synergy between $\mathcal{L}_{\text{align}}$ and $\mathcal{L}_{\text{domain}}$ as regularizers; Impact of $\mathcal{L}_{\text{uniform}}$
13	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{ce}}^{orig} + \mathcal{L}_{\text{triplet}}^{orig} + \mathcal{L}_{\text{domain}} + \mathcal{L}_{\text{uniform}}$	Synergy between $\mathcal{L}_{\text{domain}}$ and $\mathcal{L}_{\text{uniform}}$ as regularizers; impact of $\mathcal{L}_{\text{align}}$
14	$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{ce}}^{orig} + \mathcal{L}_{\text{triplet}}^{orig} + \mathcal{L}_{\text{align}} + \mathcal{L}_{\text{uniform}} + \mathcal{L}_{\text{domain}}$	Synergy between $\mathcal{L}_{\text{align}}$, $\mathcal{L}_{\text{uniform}}$ and $\mathcal{L}_{\text{domain}}$ as regularization terms on augmented images

- Do the first five and see how it is
- If triplet loss curve looks good, go to 5-10
- Do the ablation from BAU on Finsler
- Reduce the range of the hyperparameters of alpha
    - 0,, (0.01), 0.1, (0.3), 0.5
    - If 0.1, 0.5 are fluctuating, we stop there
    - Above 0.5 does not give good results
    - Ri

    - Right now its difficult to see how Finsler synergizes with BAU loss terms
    - Learnable alpha? – add hyperparameter here
    - One number output with sigmoid activation to constrain it from 0 to 1.

Finsler Alpha Sweep with Mean and Variance



Theory: Because TripletLoss relies on argmax to find the hardest negative, small numerical noise causes it to select different negatives, diverging the training trajectory significantly across runs (high variance). -> Ablate loss terms and see impact on variance

- Each alpha value ran 5 times in total, the values of mAP and Rank-1 retrieval averaged across all five runs to yield the mean
- The variances of both performance metrics are large -> might suggests instability in the model training
- Even different runs of the baseline (alpha = None) yields varying values of mAP and Rank-1
- Paper reports this value as 42.8 and 43.9?

- If there is a five runs you can plot them directly
- Stick to five is okay

- We have to see how baseline averaged

- What is the difference between rank-1 and mAP?
- Having alpha as a learnable hyperparameter might be useful here, learn perhaps a non-uniform alpha

Finsler Alpha Sweep with Mean and Variance

Alpha	mAP	mAP_mean	mAP_std	Rank-1	Rank-1_mean	Rank-1_std
None	42.2			42.1		
0	41.3			42.1		
0	42.6			42.4		
0	42.1	42.16	0.517687164	42.6	42.38	0.396232255
0	42.3			41.9		
0	42.5			42.9		
0.1	42.1			42.2		
0.1	42.1			42.4		
0.1	41.9	42.26	0.427784993	41.9	42.7	0.98488578
0.1	42.2			42.6		
0.1	43			44.4		
0.5	41.8			42.7		
0.5	42.4			42.9		
0.5	42.1	42.16	0.250998008	42.1	42.7	0.40620192
0.5	42.4			43.2		
0.5	42.1			42.6		

Re-running baseline gives a different value -> Perhaps we were just lucky this time?

Next: Copy a “clean” version of the BAU repository and run the exact same hyperparameter config 5 times; compute mean and variances

Same RNG experiment but in a tabular form

Variance / Discrepancy from Baseline not from Finsler term

```python
if alpha is not None:
    assert alpha < 1        # Sanity Check
    y_last = y[:, -1].unsqueeze(0)  # (1, n)
    x_last = x[:, -1].unsqueeze(1)  # (m, 1)
    finsler_term = (y_last - x_last) * alpha
    assert finsler_term.shape == dist.shape, "Finsler term shape mismatch"

    # DEBUG: Sanity check for alpha=0
    if alpha == 0:
        # If everything is correct, dist + finsler_term should be identical to dist, or at least
        # within a tiny float tolerarnce.

        # 1. Check for NaNs (e.g., if inputs had Inf, Inf * 0 = NaN)
        if torch.isnan(finsler_term).any():
            print(f"[DEBUG CRITICAL] alpha=0 but Finsler term contains NaNs! Check inputs for Infinity.")

        # 2. Check for values > 0 (should all be zeros)
        if not torch.allclose(finsler_term, torch.zeros_like(finsler_term)):
            max_val = finsler_term.abs().max().item()
            print(f"[DEBUG CRITICAL] alpha=0 but Finsler term is not zero! Max absolute value: {max_val}")

        # 3. Direct comparison of before/after distance matrices
        new_dist = dist + finsler_term
        diff = (dist - new_dist).abs()
        if diff.max() > 1e-7:  # Tolerance for float32 additions
            print(f"[DEBUG CRITICAL] alpha=0 distance matrix mismatch! Max diff: {diff.max().item()}")

        print(f"[DEBUG] alpha=0 sanity check completed.")
    dist = dist + finsler_term
```

log.txt

logs > finsler > job_1481113_alpha0.0 > log.txt

DEBUG CRITICAL No results

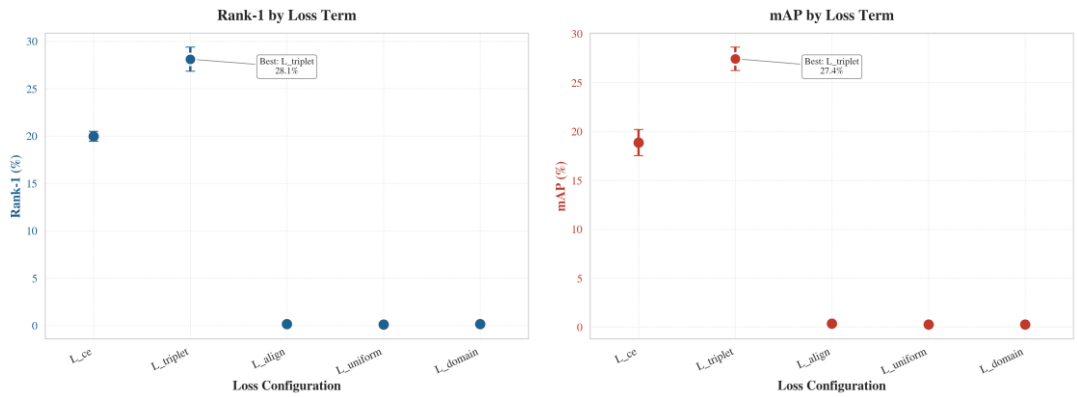
```
Trainer constructed
Starting training loop for 60 epochs
Epoch 1/60 - starting
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
```

- Placed sanity check hooks in the computation of the finsler term, but only when alpha == 0
- Conducted three checks, none of them are called as evident in the log file on the right

- If we just change the last component, the meaning of alpha might be worthless if the vector is large
- Learning alpha adds a degree of freedom to the model, which introduces real dissimilarity in the model
- When we fix the alpha, we are rescaling the difference between the last components
- Beyond learning the alpha, we might learn omega (magnitude and direction)

- Add alpha as a parameter and learn it
- Use a pretrained mode and initialize with it (alpha=0)
- Add some constraints in computing alpha (apply a sigmoid to the scalar)

Individual Impact of Loss Terms on Variances in mAP and Rank-1 (alpha=0.0)



Alternative: Disable Triple Loss but keep the others

- Ablated all the loss terms, each run only has one single loss term running
- Confirms that alignment loss, uniform loss and domain loss have zero performance and zero variance when used in isolation (inert)
    - Regularization terms that require a feature structure
    - Not the source of instability

- Argmax instability supported by Rank-1 data
- But cross-entropy actually introduces more variance to the mAP score than triplet

- Keep something from the task when doing these experiments
- Keep cross entropy loss all the way?
- All the three regularization terms would work as

- Learning alpha needs to be at least as good (performance does not drop)

Individual Impact of Loss Terms on Variances in mAP and Rank-1 (alpha=0.0)

LossName	mAP	Rank-1	mAP_mean	mAP_std	Rank-1_mean	Rank-1_std
L_ce	17.4	19.4	18.86666667	1.331665624	19.96666667	0.513160144
L_ce	20	20.1				
L_ce	19.2	20.4				
L_triplet	27.8	28.4	27.43333333	1.193035345	28.1	1.276714533
L_triplet	28.4	29.2				
L_triplet	26.1	26.7				
L_align	0.3	0.1	0.366666667	0.115470054	0.166666667	0.115470054
L_align	0.3	0.1				
L_align	0.5	0.3				
L_uniform	0.2	0.1	0.266666667	0.057735027	0.1	0
L_uniform	0.3	0.1				
L_uniform	0.3	0.1				
L_domain	0.3	0.2	0.266666667	0.057735027	0.166666667	0.057735027
L_domain	0.2	0.1				
L_domain	0.3	0.2				

Next Steps

1. Copy a “clean” version of the BAU repository and run the exact same hyperparameter config 5 times; compute mean and variances (seeing baseline std aligns with alpha=0)
2. Find a configuration for learnable alpha where the performance is better or same
   - Higher chance for finsler to be better if the dataset suits finsler
   - Asymmetry between classes, persons, clothes
   - Draw the network that I used to learn alpha / formula to convey to Simon and Thomas on what I am doing
   - Initialize from scratch (alpha=0.0) and from pretrained models (BAU Baseline)

3. Implement learnable omega
4. Continue working on the visualisations

- Ran three main experiments: CE Loss, Triplet Loss and Alignment Loss